Status	Finished
Started	Monday, 26 October 2026, 5:38 PM
Completed	Monday, 26 October 2026, 7:10 PM
Duration	1 hour 31 mins
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
filename = input().strip()
line_number = int(input().strip())

try:
    with open(filename, "r") as f:
        for i, line in enumerate(f, start=1):
            if i == line_number:
                print(line.rstrip("\n"))
                break
        else:
            print("Line number out of range.")
except FileNotFoundError:
    print("File not found.")
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
from collections import Counter
import re
import sys


def count_word_frequencies():
    # Read raw input from stdin
    input_text = sys.stdin.read().strip()
    if not input_text:
        return

    # Check if input is a filename; otherwise, use as direct text content
    try:
        with open(input_text, "r") as file:
            content = file.read()
    except (FileNotFoundError, OSError):
        content = input_text

    # Extract words, convert to lowercase, and strip punctuation
    raw_words = content.split()
    cleaned_words = [re.sub(r"^\W+|\W+$", "", word).lower() for word in raw_words]
    cleaned_words = [word for word in cleaned_words if word]

    # Count frequencies and sort alphabetically by word
    word_counts = Counter(cleaned_words)
    sorted_counts = sorted(word_counts.items())

    # Format output string
    output_lines = [f"{word}: {count}" for word, count in sorted_counts]
    output_content = "\n".join(output_lines) + "\n"

    # Write result to output.txt
    with open("output.txt", "w") as out_file:
        out_file.write(output_content)


if __name__ == "__main__":
    count_word_frequencies()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys


def count_total_words():
    # Read the filename from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    filename = input_data[0]

    # Read file content and count whitespace-separated words
    with open(filename, "r") as file:
        content = file.read()

    words = content.split()
    total_words = len(words)

    # Output the required format
    print(f"Total words: {total_words}")


if __name__ == "__main__":
    count_total_words()
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys


def find_longest_word():

    input_data = sys.stdin.read().split()
    if not input_data:
        return

    filename = input_data[0]


    with open(filename, "r") as file:
        words = file.read().split()

    if words:

        longest_word = max(words, key=len)
        print(f"Longest word: {longest_word}")


if __name__ == "__main__":
    find_longest_word()
```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys


def reverse_line_content():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    target_line = int(input_data[1].strip())

    with open(filename, "r") as file:
        lines = file.readlines()

    if 1 <= target_line <= len(lines):
        idx = target_line - 1
        line = lines[idx].rstrip("\r\n")

        # Separate trailing period/punctuation if present
        if line.endswith("."):
            core_text = line[:-1]
            reversed_line = core_text[::-1] + "."
        else:
            reversed_line = line[::-1]

        lines[idx] = reversed_line + "\n"

    with open("output.txt", "w") as out_file:
        out_file.writelines(lines)
```

```
33 if __name__ == "__main__":
34     reverse_line_content()
35
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys


def replace_line_in_file():
    # Read all inputs passed by the tester
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 3:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    # Read the target file content
    with open(filename, "r") as file:
        lines = file.readlines()

    # Replace the target line (1-based index)
    if 1 <= line_number <= len(lines):
        lines[line_number - 1] = new_content.rstrip("\r\n") + "\n"

    # Save the result to output.txt
    with open("output.txt", "w") as out_file:
        out_file.writelines(lines)


if __name__ == "__main__":
    replace_line_in_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import re
import sys


def find_palindromes():
    # Read the filename from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    filename = input_data[0]

    # Open target file and extract words
    with open(filename, "r") as file:
        content = file.read()

    # Extract clean words, stripping punctuation and converting to lowercase
    raw_words = content.split()
    palindromes = []

    for raw_word in raw_words:
        cleaned_word = re.sub(r"^\W+|\W+$", "", raw_word).lower()
        if cleaned_word and len(cleaned_word) > 1:
            if cleaned_word == cleaned_word[::-1]:
                palindromes.append(cleaned_word)

    # Write output to output.txt
    output_content = "\n".join(palindromes)
    if output_content:
        output_content += "\n"

    with open("output.txt", "w") as out_file:
        out_file.write(output_content)


if __name__ == "__main__":
    find_palindromes()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys


def insert_line_at_position():
    # Read all input lines from stdin
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 3:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_line_content = input_data[2].rstrip("\r\n")

    # Read target file contents
    with open(filename, "r") as file:
        raw_lines = file.readlines()

    # Ensure all existing lines end with a newline character
    lines = [line if line.endswith("\n") else line + "\n" for line in raw_lines]

    # Convert 1-based index to 0-based insertion index
    insert_idx = line_number - 1

    # Insert new line shifting subsequent lines down
    if insert_idx >= len(lines):
        lines.append(new_line_content + "\n")
    else:
```

```
        lines.insert(insert_idx, new_line_content + "\n")

    # Write modified contents to output.txt
    with open("output.txt", "w") as out_file:
        out_file.writelines(lines)


if __name__ == "__main__":
    insert_line_at_position()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import sys


def copy_file_contents():

    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    source_filename = input_lines[0].strip()
    destination_filename = input_lines[1].strip()


    with open(source_filename, "r") as src_file:
        content = src_file.read()

    with open(destination_filename, "w") as dest_file:
        dest_file.write(content)


if __name__ == "__main__":
    copy_file_contents()
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys


def delete_line_from_file():
    # Read filename and target line number from standard input
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    # Read original contents from target file
    with open(filename, "r") as file:
        lines = file.readlines()

    # Convert 1-based index to 0-based index and delete the line
    if 1 <= line_number <= len(lines):
        del lines[line_number - 1]

    # Write modified contents to output.txt
    with open("output.txt", "w") as out_file:
        out_file.writelines(lines)


if __name__ == "__main__":
    delete_line_from_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.